Neural Network-Based European Option Pricing under Black-Scholes SDE Dynamics

Project group

May 28, 2026

Abstract

This project studies whether a feed-forward neural network can accurately approximate the Black-Scholes pricing function for European call options. The problem is addressed in a controlled setting: data are generated synthetically from the analytical Black-Scholes formula, while Monte Carlo simulation is used as a numerical benchmark. The neural network is trained in a supervised learning framework to learn the mapping from market and contract parameters to the theoretical option price. The goal is not to build a real trading system, but to evaluate the role of a neural network as a pricing surrogate in a mathematically well-defined financial setting.

Contents

1 Introduction

Derivative pricing is one of the classical problems in quantitative finance. For European options, the Black-Scholes model provides a closed-form price under precise assumptions on the dynamics of the underlying asset, the market, and volatility [1]. The availability of an analytical solution makes Black-Scholes an ideal controlled environment for evaluating the accuracy of numerical methods and machine learning models.

The research question of the project is:

> Can a feed-forward neural network accurately approximate the Black-Scholes pricing function for European call options?

The project compares three approaches:

- the analytical Black-Scholes formula, used as the ground truth;
- Monte Carlo simulation, used as a numerical benchmark;
- a feed-forward neural network, used as a supervised learning model and pricing surrogate.

The main objective is not to replace Black-Scholes in a setting where a closed-form formula is available, but to verify whether a neural network can learn a known pricing function. This is a useful first step for understanding neural pricing methods in more complex settings, where analytical formulas are unavailable or computationally expensive.

2 Theoretical Framework

2.1 Underlying Asset Dynamics

In the Black-Scholes model, the price of the underlying asset follows a geometric Brownian motion under the risk-neutral measure. This formulation is standard in continuous-time financial market modeling [4]:

$$dS_t = rS_t dt + \sigma S_t dW_t, \tag{2.1}$$

where S_t is the asset price at time t, r is the risk-free rate, σ is the constant volatility, and W_t is a standard Brownian motion.

The solution of the SDE is:

$$S_T = S_0 \exp\left[\left(r - \frac{1}{2}\sigma^2\right)T + \sigma\sqrt{T}Z\right], \qquad Z \sim \mathcal{N}(0,1). \tag{2.2}$$

2.2 European Call Option

A European call option with strike K and maturity T has payoff:

$$\max(S_T - K, 0). \tag{2.3}$$

Under risk-neutral pricing, its value is the discounted expected payoff:

$$C = e^{-rT}\mathbb{E}^{\mathbb{Q}}\left[\max(S_T - K, 0)\right]. \tag{2.4}$$

2.3 Black-Scholes Formula

The analytical price of a European call option is:

$$C_{BS} = S_0 N(d_1) - Ke^{-rT}N(d_2), \tag{2.5}$$

where $N(\cdot)$ is the cumulative distribution function of a standard normal random variable and:

$$d_1 = \frac{\ln(S_0/K) + \left(r + \frac{1}{2}\sigma^2\right)T}{\sigma\sqrt{T}}, \tag{2.6}$$

$$d_2 = d_1 - \sigma\sqrt{T}. \tag{2.7}$$

In this project, C_{BS} is used both as the supervised learning target and as the reference value for evaluation.

2.4 Monte Carlo Pricing

Monte Carlo simulation approximates the risk-neutral expectation by generating many realizations of S_T:

$$\hat{C}_{MC} = e^{-rT} \frac{1}{M} \sum_{i=1}^{M} \max(S_T^{(i)} - K, 0). \tag{2.8}$$

For sufficiently large M, $\hat{C}_{MC}$ converges to C_{BS}. In this project, Monte Carlo is used as a numerical benchmark and as a comparison point for the neural network.

3 Methodology

3.1 Synthetic Dataset

The dataset is generated by randomly sampling option and market parameters. Each observation initially contains the primitive Black-Scholes input variables:

$$x = (S_0, K, T, r, \sigma), \tag{3.1}$$

and the target is:

$$y = C_{BS}(S_0, K, T, r, \sigma). \tag{3.2}$$

The initial parameter ranges are:

Variable	Description	Range
S_0	initial underlying price	$[50, 150]$
K	strike price	$[50, 150]$
T	maturity in years	$[0.1, 2]$
r	risk-free rate	$[0, 0.05]$
σ	volatility	$[0.1, 0.6]$

Table 3.1: Parameter ranges used to generate the synthetic dataset.

Using synthetic data makes the problem controlled and allows the neural network error to be evaluated directly against the exact target function. For the final model, the input vector is augmented with the engineered moneyness feature:

$$x_{\text{final}} = (S_0, K, T, r, \sigma, S_0/K). \tag{3.3}$$

Moneyness does not add information beyond S_0 and K, but it exposes a financially meaningful relation directly to the neural network.

3.2 Neural Network

The model is a feed-forward neural network for regression. This choice is motivated by the fact that sufficiently large feed-forward networks are universal function approximators under general assumptions [2, 3]. The baseline architecture is:

$$5 \to 64 \to 64 \to 32 \to 1, \tag{3.4}$$

with ReLU activation functions in the hidden layers. After the intermediate experiments, the selected final configuration uses six input features, including moneyness, and SiLU hidden activations:

$$6 \to 64 \to 64 \to 32 \to 1. \tag{3.5}$$

The network approximates the function:

$$f_\theta(S_0, K, T, r, \sigma, S_0/K) \approx C_{BS}. \tag{3.6}$$

The optimized loss is the mean squared error:

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^{n} \left(f_\theta(x_i) - y_i \right)^2 . \tag{3.7}$$

3.3 Training and Validation

The data are split into training, validation, and test sets. Inputs are standardized using `StandardScaler`; the target is also scaled during training to improve numerical stability and convergence. The network is trained with Adam, early stopping, and a fixed random seed for reproducibility.

3.4 Metrics

The main metrics are:

- Mean Absolute Error (MAE);
- Root Mean Squared Error (RMSE);
- coefficient of determination R^2;
- MAPE computed only on prices greater than 1, to avoid instability when the theoretical option price is close to zero.

4 Implementation

The project is implemented in Python with a modular structure. The main libraries are NumPy, pandas, SciPy, scikit-learn, matplotlib, and PyTorch.

4.1 Code Structure

Listing 4.1: Main project structure.

```
src/nn_option_pricing/
    black_scholes.py   # analytical formula and payoff
    monte_carlo.py     # efficient Monte Carlo simulation
    dataset.py         # synthetic data generation and storage
    model.py           # PyTorch feed-forward neural network
    training.py        # training loop, scaling, early stopping
    evaluation.py      # regression metrics
    plots.py           # visualizations
    pipeline.py        # end-to-end experiment
scripts/
    run_experiment.py     # command-line experiment entry point
    benchmark_runtime.py  # runtime benchmark entry point
```

4.2 Computational Efficiency

The Black-Scholes formula and the dataset generation procedure are implemented in vectorized form with NumPy. The Monte Carlo method uses the exact terminal distribution of the geometric Brownian motion at time T, avoiding unnecessary time discretization. Moreover, Monte Carlo simulation is batched both over options and over simulated paths, which keeps memory usage under control.

The neural network training uses PyTorch `DataLoader`, mini-batch gradient descent, input standardization, and target scaling. These choices make the code suitable both for quick development runs and for larger experiments.

4.3 Reproducibility

To make the experiments reproducible, the project fixes the random seeds of NumPy, Python, and PyTorch. The artifacts produced by an experiment include:

- the generated synthetic dataset;
- the model checkpoint;
- the input scaler;
- the target scaler;
- metrics in JSON format;
- diagnostic plots.

5 Experiments

5.1 Final Experiment Design

The final experiment was designed to evaluate the neural network as a supervised surrogate of the Black-Scholes European call pricing function on a sufficiently large synthetic dataset. The analytical Black-Scholes formula is used as the reference target, while Monte Carlo simulation is used as a numerical benchmark on a deterministic subset of the test set.

The final run used 100,000 synthetic option contracts sampled from the parameter ranges defined in the methodology. The selected neural network configuration includes moneyness S_0/K as an engineered input feature and uses the SiLU activation function in the hidden layers. The neural network was trained for up to 200 epochs with early stopping, using mini-batches of size 1024. The Monte Carlo benchmark used 50,000 simulated paths on 512 test options.

Listing 5.1: Command used for the final experiment.

```
.venv/bin/python scripts/run_experiment.py \
  --n-samples 100000 \
  --max-epochs 200 \
  --batch-size 1024 \
  --mc-n-paths 50000 \
  --mc-evaluation-samples 512 \
  --feature-set with_moneyness \
  --activation silu \
  --seed 42 \
  --data-dir data/final_improved \
  --output-dir outputs/final_improved
```

5.2 Reproducible Artifacts

The complete generated dataset, trained model checkpoint, and fitted scalers are stored under `data/final_improved/` and `outputs/final_improved/`. These directories are ignored by Git because they can be reproduced from the code and experiment configuration.

The selected final artifacts used in the report are tracked in `results/final/`. This directory contains:

- the experiment configuration snapshot;
- neural network metrics;
- Monte Carlo benchmark metrics;
- diagnostic figures;
- a short verification note reporting the final test and documentation checks.

5.3 Evaluation Protocol

The neural network is evaluated on a held-out test set that is not used during training or validation. The target prices are analytical Black-Scholes prices, so the reported errors measure how accurately the network approximates the known pricing function.

The Monte Carlo benchmark is evaluated against the same analytical Black-Scholes reference, but only on a deterministic subset of 512 test options. This restriction keeps the numerical benchmark computationally bounded while still providing a useful comparison with a classical simulation-based method.

The main metrics are mean absolute error, root mean squared error, coefficient of determination, and MAPE computed only on prices greater than 1. The MAPE restriction avoids unstable percentage errors for options whose theoretical price is close to zero.

5.4 Support Vector Regression Baseline

As an additional classical machine learning baseline, the project includes a reduced-scale Support Vector Regression experiment with an RBF kernel. This benchmark is not intended to replace the main neural network experiment. Instead, it provides a useful comparison with a non-neural supervised regression method.

The SVR benchmark is evaluated on smaller synthetic datasets because kernel methods scale less favorably than neural networks as the number of training observations increases. For this reason, the experiment uses 5,000 synthetic samples per run and is repeated over three random seeds. The same engineered feature set used in the final neural network experiment, including moneyness S_0/K, is used for the SVR baseline. Both input features and target prices are standardized before fitting, and predictions are transformed back to original price units before evaluation.

Listing 5.2: Command used for the SVR benchmark.

```
.venv/bin/python scripts/run_svr_benchmark.py \
  --n-samples 5000 \
  --seeds 11 42 73 \
  --feature-set with_moneyness \
  --output-dir results/experiments/svr_benchmark
```

5.5 Diagnostic Visualizations

The experiment produces the following diagnostic plots:

- training loss and validation loss;
- comparison between the Black-Scholes price and the neural network prediction;
- distribution of prediction errors;
- absolute error as a function of moneyness S_0/K;
- absolute error as a function of maturity T;
- absolute error as a function of volatility σ.

6 Results

6.1 Neural Network Approximation

The final neural network results on the held-out test set are reported in Table 6.1.

Metric	Value
MAE	0.04290
RMSE	0.06208
R^2	0.999993
MAPE, prices > 1	0.4803%

Table 6.1: Final neural network metrics against analytical Black-Scholes prices.

The results show that the feed-forward neural network with moneyness as an engineered feature and SiLU hidden activations approximates the Black-Scholes pricing function with very high accuracy on the sampled parameter domain. The coefficient of determination is close to one, while MAE and RMSE are small in absolute price units. This supports the main research question: in this controlled setting, a neural network can learn the Black-Scholes pricing map effectively and can be interpreted as a fast pricing surrogate after training.

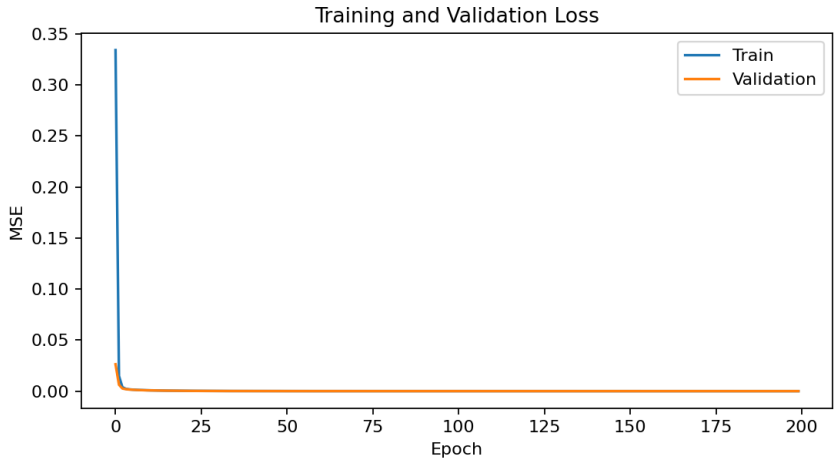


Figure 6.1: Training loss and validation loss for the final experiment.

Figure 6.1 shows that the optimization process remains stable. The validation curve follows the training curve closely, suggesting that the model does not simply memorize the training set.

Figure 6.2: Analytical Black-Scholes prices against neural network predictions.

Figure 6.2 confirms that predicted prices are tightly aligned with analytical prices across the observed price range.

6.2 Error Diagnostics

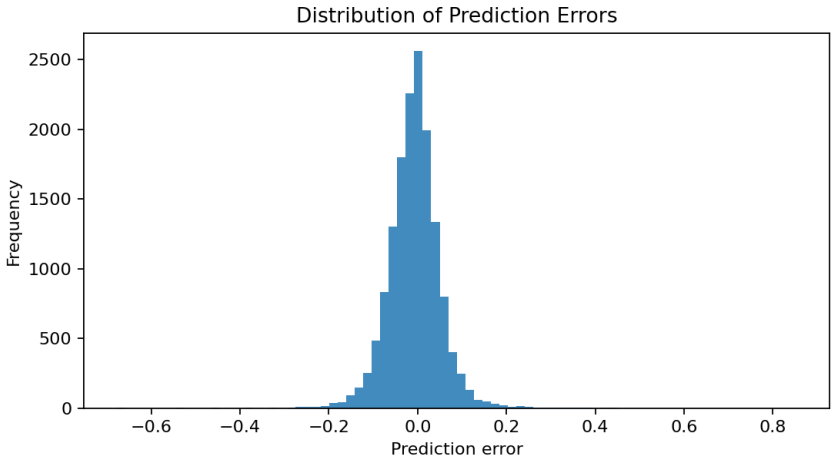
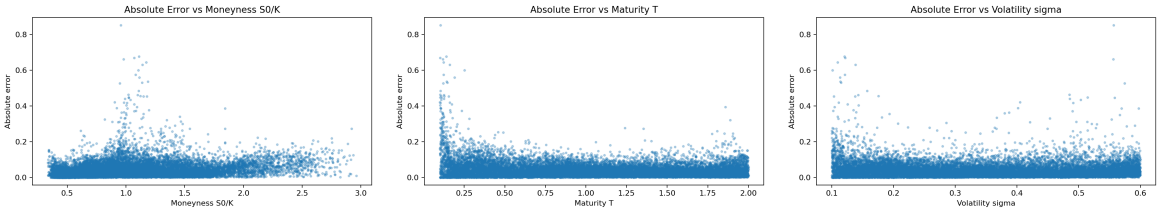


Figure 6.3: Distribution of neural network prediction errors.

The error distribution in Figure 6.3 is concentrated near zero, which is consistent with the low MAE and RMSE values reported in Table 6.1.



(a) Moneyness (b) Maturity (c) Volatility

Figure 6.4: Absolute neural network error against selected financial features.

Figure 6.4 is used to inspect whether the approximation error changes systematically across the parameter space. This is important because aggregate metrics alone may hide localized weaknesses, especially around different moneyness regimes, short maturities, or high-volatility regions.

6.3 Monte Carlo Benchmark

The Monte Carlo benchmark is evaluated against the analytical Black-Scholes price on 512 deterministic test options. The results are shown in Table 6.2.

Metric	Value
MAE	0.08882
RMSE	0.15790
R^2	0.999951
MAPE, prices > 1	0.6483%

Table 6.2: Final Monte Carlo metrics against analytical Black-Scholes prices.

The Monte Carlo benchmark also achieves high accuracy, as expected when the number of simulated paths is large. However, it remains a simulation-based estimator and therefore has sampling error. The neural network, in contrast, produces deterministic forward-pass predictions once trained. This highlights the main computational role of the network: it does not replace the analytical Black-Scholes formula in this setting, but it demonstrates how a learned surrogate can approximate a pricing map efficiently.

6.4 Runtime Benchmark

The runtime benchmark separates the initial neural network training cost from pricing time after training. Using the improved final configuration, training required approximately 359 seconds in the benchmark environment. Once trained, the neural network priced 1,000 options in approximately 0.0319 seconds on average. The Monte Carlo benchmark with 20,000 paths required approximately 1.2824 seconds for the same number of options, while the vectorized analytical Black-Scholes formula required approximately 0.00091 seconds.

Method	Mean time for 1,000 options	Mean options/s
Black-Scholes formula	0.00091 s	1,101,094.82
Neural network inference	0.03190 s	31,350.77
Monte Carlo, 20,000 paths	1.28235 s	779.82

Table 6.3: Runtime benchmark for analytical, neural-network, and Monte Carlo pricing.

As expected, the analytical Black-Scholes formula is the fastest method when available. The relevant surrogate-pricing comparison is therefore between neural network inference and Monte Carlo simulation. In this benchmark, neural inference is roughly 40 times faster than Monte Carlo after the model has been trained.

6.5 Support Vector Regression Baseline

The reduced-scale SVR benchmark provides an additional comparison with a classical supervised regression model. The experiment was repeated over three random seeds, using 5,000 synthetic contracts per run and the same engineered moneyness feature used by the final neural network. Aggregate results are reported in Table 6.4.

Metric	Mean	Standard deviation
MAE	0.15088	0.00732
RMSE	0.23100	0.00592
R^2	0.999905	0.000005
MAPE, prices > 1	1.8253%	0.2422%
Fit time	12.3198 s	0.7977 s
Prediction time	0.0301 s	0.0029 s

Table 6.4: Reduced-scale SVR benchmark aggregated over three random seeds.

The SVR baseline also approximates the Black-Scholes pricing function well on reduced datasets, as indicated by its high R^2. However, its absolute error is higher than that of the final neural network configuration. This supports the interpretation of SVR as a useful classical baseline, while the neural network remains the main scalable surrogate model in the project.

6.6 Discussion

The final results indicate that the neural network performs better than the Monte Carlo benchmark in terms of MAE, RMSE, and MAPE on the reported experiment. This comparison should be interpreted carefully because the two methods are evaluated differently: the neural network is evaluated on the full held-out test set, whereas Monte Carlo is evaluated on a deterministic subset due to computational cost.

The SVR benchmark should also be interpreted with caution because it is evaluated on reduced datasets. Its purpose is to provide an additional classical machine learning reference point, not to change the main experimental comparison.

The most important conclusion is qualitative rather than competitive. In a controlled Black-Scholes environment, a feed-forward network can learn the mapping from option and market parameters to theoretical prices with high accuracy. This validates the use of the Black-Scholes setting as a clean benchmark for studying neural-network pricing surrogates before moving to more complex models.

7 Conclusions and Next Steps

The final experiment confirms that a feed-forward neural network can approximate the Black-Scholes pricing function for European call options with high accuracy in a controlled synthetic environment. With 100,000 synthetic contracts, the network achieves an MAE of 0.04290, an RMSE of 0.06208, and an R^2 of 0.999993 on the held-out test set. These results answer the research question positively within the assumptions and parameter ranges considered in the project.

The comparison with Monte Carlo also clarifies the role of the neural network. Monte Carlo is a classical numerical estimator and remains useful as a benchmark, while the trained neural network acts as a deterministic pricing surrogate. The purpose of the network is therefore not to replace the analytical Black-Scholes formula, but to demonstrate that a supervised model can accurately learn a well-defined pricing map. The runtime benchmark further supports the surrogate interpretation: after training, neural inference is substantially faster than Monte Carlo simulation for repeated pricing queries. The additional SVR experiment provides a classical machine learning reference point on reduced datasets and supports the choice of the neural network as the main scalable surrogate model.

The main limitations are:

- the data are synthetic and generated under Black-Scholes assumptions;
- only European call options are considered;
- the target pricing function is known analytically;
- generalization outside the sampled parameter ranges is not guaranteed.

Natural next steps are:

- compare additional neural network architectures beyond the selected MLP;
- tune and evaluate further classical machine learning baselines on controlled reduced datasets;
- study the error in specific regions of the parameter space, for instance deep in-the-money or out-of-the-money options;
- extend the framework to stochastic volatility models such as Heston;
- estimate Greeks using either automatic differentiation or finite differences.

These extensions are outside the scope of the current implementation, but they represent natural directions for generalizing the approach beyond the Black-Scholes benchmark.

Bibliography

[1] Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973.

[2] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2:303–314, 1989.

[3] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.

[4] Steven E. Shreve. *Stochastic Calculus for Finance II: Continuous-Time Models*. Springer, 2004.